

From Curricular Signals to Exploratory Tasks: A Curriculum and Pedagogical Design Case Study for Integrating CAS in Upper Secondary Education

Alexandre Trocado^{1,2}, José Manuel Dos Santos³, Zsolt Lavicza²

¹ CIDMA - Center for Research and Development in Mathematics and Applications,
Rio Tinto, Portugal

² Johannes Kepler University Linz, Austria
mail@alexandretrocado.com, zsolt.lavicza@jku.at

³ University of Coimbra, Portugal
dossantos@uc.pt

Key words: *Computer Algebra Systems; GeoGebra CAS; Python/SymPy; Upper Secondary Education; Curriculum Analysis; White-Box/Black-Box; Computational Thinking; Pedagogical Design*

How can national curricula that generically endorse “the use of technology” be converted into concrete pedagogical frameworks for integrating Computer Algebra Systems in upper secondary mathematics? This talk addresses this question through a curriculum and pedagogical design case study grounded in the Portuguese *Aprendizagens Essenciais* (Essential Learning Standards) for Mathematics A — the science-track upper secondary course (grades 10–12, 2023 revision) [6].

Curricular analysis and four signals. A systematic document analysis of the three curricular documents, guided by five analytical dimensions (explicit technology reference, nature of targeted competence, CAS/symbolic potential, white-box/black-box suitability, and content anticipation potential), reveals four curricular signals that jointly support a pedagogically grounded CAS integration. The first is an *implicit legitimation* of symbolic computation: the curriculum refers to GeoGebra and “symbolic computation” as technological possibilities, yet the *Ações Estratégicas de Ensino do Professor* that accompany each topic remain confined to graphical and numerical use [6] — a pattern consistent with the international gap documented by Clark-Wilson et al. [2]. The second is a *concentration of CAS potential on core topics*: functions, derivatives, sequences, analytic geometry, and complex numbers — precisely the topics with the highest curricular weight — score highest on symbolic potential and white-box/black-box suitability. The third is a *vertical anticipation chain*: CAS-mediated exploration at each grade level opens mathematically coherent windows onto concepts from subsequent years or from university mathematics (e.g., rates of change in grade 10 anticipating derivatives in grade 11; partial sums anticipating integrals; n -th roots of unity anticipating complex analysis). The fourth is *Python and Computational Thinking as a gateway*: the curriculum explicitly prescribes Python across all three years, but the examples provided are exclusively numerical and procedural; we propose the SymPy library [7] as the natural symbolic extension of this existing curricular strand.

Design framework and illustrative tasks. Building on these signals and on the theoretical foundations of the white-box/black-box principle [1] — adapted here for the secondary level following Lagrange [5] —, instrumental genesis [4], and the integration of Computational Thinking in mathematics [8], we derive five design principles — anchor integration in existing curricular openings; sequence exploration before automation (white-box $\rightarrow$ black-box); combine visual and programmatic CAS; use anticipation only when mathematically coherent; privilege interpretation over routine execution — that organise a three-stage integration framework aligned with the three years of Mathematics A. In **Stage 1** (grade 10), the CAS serves as a transparent tool for guided symbolic exploration — for instance, students discover the factor–root relationship by

using **Factor** and **Solve** in GeoGebra CAS before the formal treatment. In **Stage 2** (grade 11), the CAS generates conjectures that the student then formalises: derivatives of $x^2, x^3, x^4, \dots$ are computed via **diff** in SymPy so that the power rule nx^{n-1} emerges inductively; after formulating and verifying the rules manually, the student transitions to legitimate black-box automation. In **Stage 3** (grade 12), the CAS is used autonomously for modelling and for explicit anticipation of university content, such as computing n -th roots of unity or applying De Moivre's theorem.

The framework is operationalised through five illustrative tasks, each with parallel implementations in GeoGebra CAS and Python/SymPy: (T1) Factorisation and Zeros of Polynomial Functions (grade 10); (T2) Average Rate of Change and Anticipation of the Derivative (grade 10); (T3) Sequences and Anticipation of Limits (grade 11); (T4) Construction of Derivative Rules by CAS (grade 11); and (T5) Complex Numbers, Rotations, and Roots of Unity (grade 12). Each task specifies curricular context, white-box/black-box phase, and anticipation target. The dual implementation — visual CAS and programmatic CAS — bridges mathematical exploration with the Computational Thinking practices already prescribed in the curriculum.

Contribution and transferability. Although rooted in the Portuguese case, the contribution is designed for international transferability [3]. The talk offers an analytical grid for reading any curriculum in terms of CAS integration potential, a set of explicit pedagogical design principles, and a staged framework adaptable to other national contexts. It concludes with a mixed-method quasi-experimental design for future classroom validation (grade 11, experimental vs. comparison groups, conceptual tests and process indicators), addressing the main limitation of the study while arguing that the curricular signals identified already justify the proposed framework as a starting point for implementation and research.

References

- [1] B. Buchberger. Should students learn integration rules? *SIGSAM Bulletin*, 24(1):10–17, 1990. <https://doi.org/10.1145/382276.1095228>
- [2] A. Clark-Wilson, O. Robutti, and M. Thomas. Teaching with digital technology. *ZDM – Math. Educ.*, 52(7):1223–1242, 2020. <https://doi.org/10.1007/s11858-020-01196-0>
- [3] B. Flyvbjerg. Five misunderstandings about case-study research. *Qualitative Inquiry*, 12(2):219–245, 2006. <https://doi.org/10.1177/1077800405284363>
- [4] D. Guin and L. Trouche. The complex process of converting tools into mathematical instruments. *Int. J. Comput. Math. Learn.*, 3(3):195–227, 1999. <https://doi.org/10.1023/A:1009549209932>
- [5] J.-B. Lagrange. Using symbolic calculators to study mathematics. In D. Guin, K. Ruthven, and L. Trouche (Eds.), *The Didactical Challenge of Symbolic Calculators*, pp. 113–135. Springer, 2005. https://doi.org/10.1007/0-387-23435-7_6
- [6] Ministério da Educação. *Aprendizagens Essenciais — Matemática A, 10.º–12.º ano (versão final)*. Lisboa: Ministério da Educação e Ciência, 2023. <https://www.dge.mec.pt/aprendizagens-essenciais-ensino-secundario>
- [7] A. Meurer et al. SymPy: Symbolic computing in Python. *PeerJ Comput. Sci.*, 3:e103, 2017. <https://doi.org/10.7717/peerj-cs.103>
- [8] D. Weintrop et al. Defining computational thinking for mathematics and science classrooms. *J. Sci. Educ. Technol.*, 25(1):127–147, 2016. <https://doi.org/10.1007/s10956-015-9581-5>